Geometric Algebra Applied to Computational Proteomics: The Specular-Grassmann-PIM (SGP) Program

Version 13.1

Contents

CHAPTER 1

Introduction

1.1 General Description

The **Specular-Grassmann-PIM (SGP) 13.1v**[1] program is a computational tool that represents protein sequences as vectors[2] in a 16-dimensional space[3], where each dimension corresponds to a specific type of interaction between pairs of amino acids[4]. Version 13.1 introduces a complete geometric algebra[5] formulation with the following operators:

- A biological metric signature[6] η that encodes the functional benefit of each interaction
- The full geometric product $v *_\eta w = (v \cdot_\eta w) + (v \wedge_\eta w)$
- Subspace projection[7] via the interior product $\lrcorner$
- Specular reflection operators[8] to detect charge inversion
- Clifford rotors[9] to measure angles in biologically relevant planes
- Commutator $[\mathbf{v}, \mathbf{w}]$ and anticommutator $\{\mathbf{v}, \mathbf{w}\}$[10]
- Streaming I/O for processing arbitrarily large datasets (151M+ sequences)

[1] SGP stands for *Specular-Grassmann-PIM*, a program that combines geometric algebra (Grassmann) with specular reflection operators to analyze protein interactions.

[2] A vector is an ordered list of numbers that represents a point or a direction in space. In this case, each vector has 16 numbers that describe a protein's interactions.

[3] A 16-dimensional space is a coordinate system with 16 different axes. Although we cannot visualize it directly, mathematics allows us to work with it just like everyday 3D space.

[4] Amino acids are the "building blocks" that form proteins. There are 20 different types in nature.

[5] Geometric algebra is a branch of mathematics that extends traditional vector algebra, enabling operations such as the geometric product that combines scalar and vector products.

[6] The metric signature is a rule that assigns signs (+ or -) to each dimension of space, indicating whether an interaction is beneficial or detrimental.

[7] A subspace is a smaller region within the total space, like a plane within 3D space.

[8] Specular reflection is an operation that "reflects" a vector as if bouncing off a mirror, swapping positive and negative charges.

[9] Clifford rotors are operators that rotate vectors within specific planes, allowing measurement of angles between proteins in concrete biological aspects.

[10] The commutator measures how much two vectors "do not commute" (i.e., the order of operation matters), while the anticommutator measures their similarity.

- Online statistics via Welford algorithm for O(1) memory usage
- Progressive reservoir sampling for memory-efficient storage
- Locality-Sensitive Hashing (LSH) for O(1) instant similarity search

1.2 Polarity Classification

Each amino acid is assigned to one of four polarity classes[11]:

Definition 1.1 (Polarity Classes)**.** Let $\mathcal{P} = \{\mathrm{P}^+, \mathrm{P}^-, \mathrm{N}, \mathrm{NP}\}$ where:

$$\begin{aligned} \mathrm{P}^+ &= \{H, K, R\} \quad \text{(positively charged)} \\ \mathrm{P}^- &= \{D, E\} \quad \text{(negatively charged)} \\ \mathrm{N} &= \{C, G, N, Q, S, T, Y\} \quad \text{(neutral polar)} \\ \mathrm{NP} &= \{A, F, I, L, M, P, V, W\} \quad \text{(nonpolar or hydrophobic)} \end{aligned} \tag{1.1}$$

[11] Polarity refers to how an amino acid interacts with water: some have electrical charge (positive or negative), others are polar without charge, and others are hydrophobic (water-repelling).

CHAPTER 2

The Core Strategy: From 150 Million Sequences to 18 Centroids

2.1 The Fundamental Insight

Before delving into the mathematical details of geometric algebra, it is essential to understand the **architectural strategy** that makes SGP capable of processing 150 million protein sequences in less than 13 hours on a standard laptop.

The key insight is simple but powerful: **we do not need to compare every protein against every other protein.** Instead, we compress each group of proteins into a compact statistical summary—the **centroid**—and perform comparisons at the group level.

2.2 The Three-Tier Data Architecture

SGP employs a three-tier data architecture that balances statistical accuracy, memory efficiency, and computational speed.

2.2.1 Tier 1: The Full Dataset (Disk Storage)

All 150 million protein sequences are stored on disk in FASTA format (approximately 170 GB uncompressed). These files are **never loaded into memory**. Instead, they are read sequentially using streaming I/O.

$$\text{Total sequences: } N_{\text{total}} = 151{,}066{,}923 \tag{2.1}$$

$$\text{Total disk space: } \sim 170 \text{ GB} \tag{2.2}$$

2.2.2 Tier 2: The Centroid (Memory, 1 per Group)

For each protein group, we compute a single centroid vector:

- One centroid = 16 floating-point numbers
- Size: 16×8 bytes = 128 bytes

- For $G = 18$ groups: 18×128 bytes = 2,304 bytes (**less than 3 KB**)

$$\boldsymbol{\mu}_G = \frac{1}{N_G} \sum_{k=1}^{N_G} \mathbf{v}^{(k)} \in \mathbb{R}^{16} \tag{2.3}$$

The centroid is the **average** PIM vector of all proteins in the group. It represents the group's typical interaction profile. This is a statistical summary, not a real protein.

2.2.3 Tier 3: The Representative Sample (Memory, 10,000 per Group)

For downstream analyses requiring individual protein vectors (e.g., finding the most similar proteins, building the LSH index), SGP stores a representative sample of 10,000 proteins per group:

- 10,000 vectors $\times$ 16 components $\times$ 8 bytes = 1.28 MB per group
- For $G = 18$ groups: 18×1.28 MB = 23 MB
- Plus headers: $\sim$18 MB
- **Total: $\sim$41 MB**

The sample is collected using **progressive reservoir sampling**, which guarantees that:

1. Every protein in the group has an equal probability of being selected.
2. The sample is representative of the entire group, even as the group grows to 150 million members.
3. The sample size remains constant regardless of the total number of proteins.

2.3 The Two-Phase Processing Strategy

2.3.1 Phase 1: Model Construction (The "Heavy" Phase)

In this phase, SGP reads all sequences once:

1. For each sequence, compute the 16-component PIM vector.
2. Update online statistics (mean and covariance) using the Welford algorithm.
3. Add the vector to the representative sample using reservoir sampling.
4. Discard the sequence (only the vector is kept, and only if selected for the sample).

Complexity: $O(N)$ time, $O(1)$ memory per group.

Result:

- 18 centroids (one per group)
- 18 samples of 10,000 vectors each (total $\sim$41 MB)
- 18 covariance matrices

2.3.2 Phase 2: Analysis (The "Fast" Phase)

With the compact representations, all subsequent analyses are extremely fast:

1. **Group comparison:** Compare centroids using geometric algebra. Only $18 \times 17/2 = 153$ comparisons.
2. **Protein classification:** Compare a query vector to 18 centroids. Only 18 comparisons.
3. **Similarity search:** Use LSH to find proteins with identical hashes. Only proteins in the same hash bucket are compared.

4. **Top-k search:** Compare the query to 10,000 sample vectors per group. Only 10,000 comparisons per group, not 150 million.

Complexity: $O(1)$ for search, $O(G)$ for classification.

2.4 Why This Strategy is Efficient

Operation	Naive Approach	SGP Approach	Speedup
Compare LUJV to all human proteins	147,506 comparisons	1 comparison (centroid vs centroid)	147,506×
Find most similar protein	150 million comparisons	10,000 comparisons (sample)	15,000×
Classify a new protein	150 million comparisons	18 comparisons (centroids)	8.3 million×
Search for similar proteins	150 million comparisons	$O(1)$ via LSH	Instant

2.5 Memory Comparison

Data	If stored completely	SGP storage
All PIM vectors (150M × 16 × 8)	19.2 GB	0 bytes (not stored)
Headers (150M × 100 bytes)	15 GB	0 bytes (not stored)
Centroids (18 groups)	N/A	2.3 KB
Representative sample (10,000 × 18 groups)	N/A	41 MB
Total	∼35 GB	∼41 MB

Compression ratio: 35 GB/41 MB ≈ 854-fold reduction in memory usage.

2.6 Verification of Complete Processing

A common concern is whether all sequences were actually processed. SGP includes built-in verification:

1. `ProcessingTracker` monitors total sequences read, valid PIM profiles, and processing rate in real time.
2. The final summary reports total sequences processed per group and overall.
3. `GroupStatistics` stores `total_processed` (all sequences) and `n_samples` (valid PIM profiles) separately.
4. The function `compute_lujv_statistics()` reads all human sequences from disk again to generate histograms, confirming the data is available.

The tracker output at the end of execution confirms that all 151,066,923 sequences were processed, even though only 10,000 per group were stored in RAM for subsequent comparisons.

2.7 Summary of the Core Strategy

- **Full dataset:** 151,066,923 sequences, 170 GB on disk (not in memory).
- **Centroids:** 18 vectors (one per group), 2.3 KB in memory.
- **Sample:** 10,000 vectors per group, 41 MB in memory.
- **Processing time:** ∼13 hours (single pass over all data).
- **Analysis time:** Sub-second to minutes (centroid and sample comparisons).

This architecture enables the analysis of 150 million sequences on a standard laptop with 16 GB RAM, without GPU acceleration or distributed computing.

CHAPTER 3

The PIM Vector Space

3.1 Vector Construction

Definition 3.1 (Pair Interaction Matrix - PIM)**.** For a protein sequence $S = (a_1, a_2, \ldots, a_L)$ with $L \geq 2$, the PIM vector[12] $\mathbf{v} \in \mathbb{R}^{16}$ has components:

$$v_k = \frac{1}{Z} \sum_{i=1}^{L-1} w(c_i, c_{i+1}) \cdot \mathbb{1}[k = \text{index}(c_i, c_{i+1})] \tag{3.1}$$

where c_i is the polarity class of amino acid a_i, $w(\cdot, \cdot)$ is a biological weight[13], and Z is the normalization constant[14]:

$$Z = \sum_{i=1}^{L-1} \sum_{k=1}^{16} w(c_i, c_{i+1}) \cdot \mathbb{1}[k = \text{index}(c_i, c_{i+1})] \tag{3.2}$$

The 16 interactions are ordered as:

$$\begin{aligned} &[0]\ \mathrm{P}^+ \to \mathrm{P}^+, \quad [1]\ \mathrm{P}^+ \to \mathrm{P}^-, \quad [2]\ \mathrm{P}^+ \to \mathrm{N}, \quad [3]\ \mathrm{P}^+ \to \mathrm{NP}, \\ &[4]\ \mathrm{P}^- \to \mathrm{P}^+, \quad [5]\ \mathrm{P}^- \to \mathrm{P}^-, \quad [6]\ \mathrm{P}^- \to \mathrm{N}, \quad [7]\ \mathrm{P}^- \to \mathrm{NP}, \\ &[8]\ \mathrm{N} \to \mathrm{P}^+, \quad [9]\ \mathrm{N} \to \mathrm{P}^-, \quad [10]\ \mathrm{N} \to \mathrm{N}, \quad [11]\ \mathrm{N} \to \mathrm{NP}, \\ &[12]\ \mathrm{NP} \to \mathrm{P}^+, \quad [13]\ \mathrm{NP} \to \mathrm{P}^-, \quad [14]\ \mathrm{NP} \to \mathrm{N}, \quad [15]\ \mathrm{NP} \to \mathrm{NP} \end{aligned} \tag{3.3}$$

3.2 Biological Weights

Definition 3.2 (Weight Function)**.** The weight $w : \mathcal{P} \times \mathcal{P} \to \mathbb{R}^+$ assigns functional importance:

[12] PIM stands for *Protein Interaction Map*, a numerical representation of the interaction pattern between adjacent amino acids in a protein.

[13] Biological weights are numbers that reflect the functional importance of each interaction type. For example, electrostatic attractions have weight 2.0 because they are very important biologically.

[14] Normalization is a process that adjusts numbers so they sum to 1, allowing comparison of proteins of different lengths.

$$w(\mathrm{P}^+, \mathrm{P}^-) = w(\mathrm{P}^-, \mathrm{P}^+) = 2.0 \quad \text{(charge attraction)} \tag{3.4}$$

$$w(\mathrm{N}, \mathrm{N}) = 1.5 \quad \text{(hydrogen bonds)} \tag{3.5}$$

$$w(\mathrm{P}^+, \mathrm{N}) = w(\mathrm{P}^-, \mathrm{N}) = w(\mathrm{N}, \mathrm{P}^+) = w(\mathrm{N}, \mathrm{P}^-) = 1.3 \tag{3.6}$$

$$w(\mathrm{NP}, \mathrm{NP}) = 1.0 \tag{3.7}$$

$$w(\mathrm{P}^+, \mathrm{P}^+) = w(\mathrm{P}^-, \mathrm{P}^-) = 0.4 \quad \text{(charge repulsion)} \tag{3.8}$$

$$w(\text{others}) = 0.7 \tag{3.9}$$

Example 3.1 (PIM Vector Calculation)**.** Consider the peptide sequence[15] "HKD" (His-Lys-Asp):

$$\text{His} \to \mathrm{P}^+, \quad \text{Lys} \to \mathrm{P}^+, \quad \text{Asp} \to \mathrm{P}^- \tag{3.10}$$

Pairs:

- H→K: $\mathrm{P}^+ \to \mathrm{P}^+$ (index 0) with weight $w = 0.4$
- K→D: $\mathrm{P}^+ \to \mathrm{P}^-$ (index 1) with weight $w = 2.0$

Unnormalized counts: $[0.4, 2.0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$, sum $= 2.4$

Normalized PIM vector:

$$\mathbf{v} = [0.1667, 0.8333, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \tag{3.11}$$

Interpretation: this protein is dominated by charge attraction ($\mathrm{P}^+ \to \mathrm{P}^-$) with 83% of its profile.

[15] A peptide is a short chain of amino acids. A complete protein can have hundreds or thousands of amino acids.

CHAPTER 4

Geometric Algebra Operators of SGP 13.1v

4.1 Biological Metric Signature

Definition 4.1 (Biological Metric η)**.** Let $\eta \in \mathbb{R}^{16}$ be the metric signature:

$$\eta_k = \begin{cases} +1 & \text{if interaction } k \text{ is beneficial} \\ -1 & \text{if interaction } k \text{ is detrimental} \\ 0 & \text{if interaction } k \text{ is neutral} \end{cases} \tag{4.1}$$

Explicitly:

$$\eta = [-1, +1, +1, 0, +1, -1, +1, 0, +1, +1, +1, 0, 0, 0, 0, +1] \tag{4.2}$$

Example 4.1 (Using the biological metric)**.** Let $\mathbf{v} = [0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.4, 0, 0, 0, 0, 0]$. Compute its metric norm[16]:

$$\begin{aligned} \mathbf{v} \cdot_\eta \mathbf{v} &= \eta_0(0.5)^2 + \eta_{10}(0.4)^2 = (-1)(0.25) + (+1)(0.16) = -0.09 \\ \|\mathbf{v}\|_\eta &= \sqrt{|-0.09|} = 0.3 \end{aligned} \tag{4.3}$$

The negative sign indicates that this protein has more detrimental components than beneficial ones.

4.2 Metric Dot Product

Definition 4.2. The metric dot product[17] for $\mathbf{v}, \mathbf{w} \in \mathbb{R}^{16}$ is:

$$\mathbf{v} \cdot_\eta \mathbf{w} = \sum_{k=0}^{15} \eta_k v_k w_k \tag{4.4}$$

[16]The metric norm is a measure of a vector's "size" that takes into account the metric signature, allowing components with sign -1 to contribute negatively.

[17]The metric dot product is an operation that multiplies two vectors component-wise, but applies the metric signature to give more or less weight according to the sign η_k.

Example 4.2 (Metric dot product)**.** Let:

$$\begin{aligned}\mathbf{v} &= [0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.4, 0, 0, 0, 0, 0] \\ \mathbf{w} &= [0.6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.3, 0, 0, 0, 0, 0]\end{aligned} \tag{4.5}$$

$$\begin{aligned}\mathbf{v} \cdot_\eta \mathbf{w} &= \eta_0(0.5)(0.6) + \eta_{10}(0.4)(0.3) \\ &= (-1)(0.3) + (+1)(0.12) = -0.18\end{aligned} \tag{4.6}$$

The negative value indicates that, although the numbers are similar, the interactions are functionally opposite.

4.3 Oriented Wedge Product

Definition 4.3 (Wedge Product with Orientation)**.** For $\mathbf{v}, \mathbf{w} \in \mathbb{R}^{16}$, the bivector[18] components for key interaction pairs are:

$$(\mathbf{v} \wedge \mathbf{w})_{ij} = v_i w_j - v_j w_i \tag{4.7}$$

The implementation computes only key pairs[19] $\mathcal{K}$:

$$\mathcal{K} = \{(0,5), (1,4), (2,6), (3,7), (10,11), (14,15)\} \tag{4.8}$$

The bivector vector is:

$$\mathbf{b} = [v_0 w_5 - v_5 w_0,\ v_1 w_4 - v_4 w_1,\ v_2 w_6 - v_6 w_2,\ v_3 w_7 - v_7 w_3,\ v_{10} w_{11} - v_{11} w_{10},\ v_{14} w_{15} - v_{15} w_{14}] \tag{4.9}$$

The normalized magnitude is:

$$\|\mathbf{v} \wedge \mathbf{w}\|_{\text{norm}} = \frac{\|\mathbf{b}\|_2}{\|\mathbf{v}\|_2 \|\mathbf{w}\|_2}, \quad \text{where } \|\mathbf{b}\|_2 = \sqrt{\sum_{j=1}^{6} b_j^2} \tag{4.10}$$

The orientation sign is $\text{sgn}(b_j)$ for the first non-zero component b_j.

Interpretation: A value of 1.0 indicates identical functional profiles, 0 indicates orthogonality, and higher values indicate greater functional similarity.

Example 4.3 (Wedge product between two proteins)**.** Let:

$$\begin{aligned}\mathbf{v} &= [0.1667, 0.8333, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \\ \mathbf{w} &= [0, 0, 0, 0, 0.5, 0, 0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0]\end{aligned} \tag{4.11}$$

Compute:

$$\begin{aligned}\|\mathbf{v}\|_2 &= \sqrt{0.1667^2 + 0.8333^2} = 0.8498 \\ \|\mathbf{w}\|_2 &= \sqrt{0.5^2 + 0.5^2} = 0.7071\end{aligned} \tag{4.12}$$

[18] A bivector is a mathematical object that represents an oriented area in space, like a parallelogram with direction. In this context, it represents the "structural difference" between two proteins.

[19] Key pairs are combinations of indices that represent biologically important interactions, such as charge repulsion or opposite charge attraction.

Bivector components:

$$\begin{aligned} b_1 &= v_0 w_5 - v_5 w_0 = (0.1667)(0) - (0)(0) = 0 \\ b_2 &= v_1 w_4 - v_4 w_1 = (0.8333)(0.5) - (0)(0) = 0.4167 \\ b_3 &= v_2 w_6 - v_6 w_2 = (0)(0.5) - (0)(0) = 0 \\ b_4 &= v_3 w_7 - v_7 w_3 = (0)(0) - (0)(0) = 0 \\ b_5 &= v_{10} w_{11} - v_{11} w_{10} = 0 \\ b_6 &= v_{14} w_{15} - v_{15} w_{14} = 0 \end{aligned} \tag{4.13}$$

$$\|\mathbf{b}\|_2 = 0.4167, \quad \|\mathbf{v} \wedge \mathbf{w}\|_{\text{norm}} = \frac{0.4167}{0.8498 \times 0.7071} = 0.693 \tag{4.14}$$

The orientation sign is positive ($b_2 > 0$). This indicates moderate structural similarity with positive orientation. Higher values indicate greater similarity.

4.4 Specular Reflection (Mirror Operator)

Definition 4.4 (Normal Vector for PP Exchange)**.** The normal vector[20] $\mathbf{n} \in \mathbb{R}^{16}$ for specular reflection is constructed from the exchange map σ:

$$\begin{aligned} \sigma(0) = 5,\ \sigma(1) = 4,\ \sigma(2) = 6,\ \sigma(3) = 7,\ \sigma(4) = 1,\ \sigma(5) = 0, \\ \sigma(6) = 2,\ \sigma(7) = 3,\ \sigma(8) = 9,\ \sigma(9) = 8,\ \sigma(10) = 10,\ \sigma(11) = 11, \\ \sigma(12) = 13,\ \sigma(13) = 12,\ \sigma(14) = 14,\ \sigma(15) = 15 \end{aligned} \tag{4.15}$$

The unnormalized normal vector is $\mathbf{n}_0$ where:

$$n_{0,i} = \sum_j (\mathbb{1}[\sigma(i) = j] - \mathbb{1}[i = j]) = \begin{cases} 1 & \text{if } \sigma(i) \neq i \text{ and } i < \sigma(i) \\ -1 & \text{if } \sigma(i) \neq i \text{ and } i > \sigma(i) \\ 0 & \text{if } \sigma(i) = i \end{cases} \tag{4.16}$$

This gives:

$$\mathbf{n}_0 = [1, 1, 1, 1, -1, -1, -1, -1, 1, -1, 0, 0, 1, -1, 0, 0] \tag{4.17}$$

Normalizing: $\|\mathbf{n}_0\|_2 = \sqrt{12} \approx 3.464$

$$\begin{aligned} \mathbf{n} = \frac{\mathbf{n}_0}{3.464} \approx \big[&0.2887,\ 0.2887,\ 0.2887,\ 0.2887,\ -0.2887,\ -0.2887,\ -0.2887,\ -0.2887, \\ &0.2887,\ -0.2887,\ 0,\ 0,\ 0.2887,\ -0.2887,\ 0,\ 0\big] \end{aligned} \tag{4.18}$$

Definition 4.5 (Specular Reflection in SGP)**.** The reflection of vector $\mathbf{v}$ across the hyperplane[21] orthogonal[22] to $\mathbf{n}$ is:

$$\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n} \tag{4.19}$$

In geometric algebra, this is $-\mathbf{nvn}$ for vectors. If the wedge product between a reflected vector and a target exceeds 0.95, the proteins are considered specular reflections.

[20] In geometry, a normal vector is a vector perpendicular to a surface or hyperplane. Here it defines the "mirror" that swaps positive and negative charges.

[21] A hyperplane is a space of dimension 15 within the 16-dimensional space, analogous to how a plane (2D) is within 3D space.

[22] Orthogonal means perpendicular. A vector orthogonal to a hyperplane forms a 90-degree angle with all directions within that hyperplane.

Example 4.4 (Specular reflection of a protein)**.** Let $\mathbf{v} = [0.5, 0, 0, 0, 0, 0.2, 0, 0, 0, 0, 0.3, 0, 0, 0, 0, 0]$.

Compute:

$$\mathbf{v} \cdot \mathbf{n} = (0.5)(0.2887) + (0.2)(-0.2887) + (0.3)(0) = 0.14435 - 0.05774 = 0.08661 \tag{4.20}$$

Then:

$$\mathbf{v}' = \mathbf{v} - 2(0.08661)\mathbf{n} = \mathbf{v} - 0.17322\mathbf{n} \tag{4.21}$$

Relevant components:

$$\begin{aligned} v_0' &= 0.5 - 0.17322(0.2887) = 0.5 - 0.05 = 0.45 \\ v_5' &= 0.2 - 0.17322(-0.2887) = 0.2 + 0.05 = 0.25 \\ v_{10}' &= 0.3 - 0.17322(0) = 0.3 \end{aligned} \tag{4.22}$$

The reflection has partially swapped the repulsions: $v_0 = 0.5 \to v_0' = 0.45$, $v_5 = 0.2 \to v_5' = 0.25$.

4.5 Interior Product

Definition 4.6 (Subspace Projection)**.** For a subspace $S \subseteq \{0, \ldots, 15\}$, the interior product[23] projects onto that subspace:

$$(\mathbf{v} \lrcorner \mathbf{e}_S)_k = \begin{cases} v_k / \sum_{j \in S} v_j & \text{if } k \in S \\ 0 & \text{otherwise} \end{cases} \tag{4.23}$$

The predefined subspaces in SGP are:

$$\begin{aligned} S_{\text{hydrophobic}} &= \{10, 15\} \\ S_{\text{charge\_repulsion}} &= \{0, 5\} \\ S_{\text{charge\_attraction}} &= \{1, 4\} \\ S_{\text{charge\_polar}} &= \{2, 3, 6, 7\} \\ S_{\text{polar}} &= \{8, 9, 10, 11\} \\ S_{\text{nonpolar}} &= \{12, 13, 14, 15\} \end{aligned} \tag{4.24}$$

Example 4.5 (Projection onto hydrophobic subspace)**.** Let $\mathbf{v} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.6, 0, 0, 0, 0, 0.4]$.

Project onto $S_{\text{hydrophobic}} = \{10, 15\}$:

$$\sum_{j \in S} v_j = 0.6 + 0.4 = 1.0 \tag{4.25}$$

Result:

$$\mathbf{v} \lrcorner \mathbf{e}_S = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.6, 0, 0, 0, 0, 0.4] \tag{4.26}$$

The projection is itself because the vector is already completely in the hydrophobic subspace. The projection magnitude is $\|\mathbf{v} \lrcorner \mathbf{e}_S\|_2 = \sqrt{0.6^2 + 0.4^2} = 0.721$.

[23] The interior product is an operation that "projects" a vector onto a subspace, extracting only the part of the vector that belongs to that subspace and renormalizing.

4.6 Commutator and Anticommutator

Definition 4.7. For vectors in geometric algebra:

$$[\mathbf{v}, \mathbf{w}] = \mathbf{v} \wedge \mathbf{w} \quad \text{(commutator)} \tag{4.27}$$

$$\{\mathbf{v}, \mathbf{w}\} = 2(\mathbf{v} \cdot \mathbf{w}) \quad \text{(anticommutator)} \tag{4.28}$$

The normalized commutator norm is:

$$\|[\mathbf{v}, \mathbf{w}]\|_{\text{norm}} = \frac{\|\mathbf{v} \wedge \mathbf{w}\|_2}{\|\mathbf{v}\|_2 \|\mathbf{w}\|_2} \tag{4.29}$$

The anticommutator similarity is:

$$S_{\text{anticomm}}(\mathbf{v}, \mathbf{w}) = \frac{|\mathbf{v} \cdot \mathbf{w}|}{\|\mathbf{v}\|_2 \|\mathbf{w}\|_2} \tag{4.30}$$

Example 4.6 (Commutator and anticommutator)**.** Let:

$$\begin{aligned} \mathbf{v} &= [0.1667, 0.8333, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \\ \mathbf{w} &= [0, 0, 0, 0, 0.5, 0, 0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0] \end{aligned} \tag{4.31}$$

From the previous example:

$$\begin{aligned} \|\mathbf{v} \wedge \mathbf{w}\|_2 &= 0.4167 \\ \|\mathbf{v}\|_2 = 0.8498, \quad \|\mathbf{w}\|_2 &= 0.7071 \\ \|[\mathbf{v}, \mathbf{w}]\|_{\text{norm}} &= \frac{0.4167}{0.8498 \times 0.7071} = 0.693 \end{aligned} \tag{4.32}$$

For the anticommutator:

$$\mathbf{v} \cdot \mathbf{w} = (0.1667)(0) + (0.8333)(0) + \ldots = 0 \tag{4.33}$$

$$S_{\text{anticomm}} = 0 \tag{4.34}$$

Interpretation: The high commutator (0.693) indicates that the vectors do not commute, i.e., they are structurally different. The zero anticommutator indicates no overlap in their components.

CHAPTER 5

The Geometric Product in SGP 13.1v

5.1 Definition

Definition 5.1 (Geometric Product with Metric)**.** The fundamental operation of geometric algebra, uniting the interior and exterior products:

$$\mathbf{v} *_\eta \mathbf{w} = (\mathbf{v} \cdot_\eta \mathbf{w}) + (\mathbf{v} \wedge_\eta \mathbf{w}) \tag{5.1}$$

where $\mathbf{v} \wedge_\eta \mathbf{w}$ is computed in the transformed metric space[24]:

$$\tilde{v}_k = \frac{v_k}{\sqrt{|\eta_k| + \varepsilon}}, \quad \tilde{w}_k = \frac{w_k}{\sqrt{|\eta_k| + \varepsilon}} \tag{5.2}$$

$$(\mathbf{v} \wedge_\eta \mathbf{w})_{ij} = \tilde{v}_i \tilde{w}_j - \tilde{v}_j \tilde{w}_i \tag{5.3}$$

Example 5.1 (Complete geometric product)**.** Let:

$$\begin{aligned} \mathbf{v} &= [0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.4, 0, 0, 0, 0, 0] \\ \mathbf{w} &= [0.6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.3, 0, 0, 0, 0, 0] \end{aligned} \tag{5.4}$$

First, compute the metric transformations. For $\eta_0 = -1$, $\sqrt{|\eta_0|} = 1$; for $\eta_{10} = +1$, $\sqrt{|\eta_{10}|} = 1$. Therefore $\tilde{\mathbf{v}} = \mathbf{v}$ and $\tilde{\mathbf{w}} = \mathbf{w}$.

$$\mathbf{v} \cdot_\eta \mathbf{w} = (-1)(0.5)(0.6) + (+1)(0.4)(0.3) = -0.3 + 0.12 = -0.18 \tag{5.5}$$

For the metric wedge product, since the only non-zero components are 0 and 10, and they do not form a pair in $\mathcal{K}$, we have $\mathbf{v} \wedge_\eta \mathbf{w} = \mathbf{0}$.

Therefore:

$$\mathbf{v} *_\eta \mathbf{w} = -0.18 + \mathbf{0} \tag{5.6}$$

[24]The transformation to metric space consists of dividing each component by the square root of the absolute value of η_k, so that components with sign -1 contribute appropriately to the wedge product.

The result is purely scalar and negative, indicating that the proteins are functionally opposite.

5.2 Decomposition Analysis

Definition 5.2 (Geometric Product Decomposition in SGP)**.** Given $\mathbf{v}, \mathbf{w} \in \mathbb{R}^{16}$, define:

$$S_{\text{func}} = \frac{|\mathbf{v} \cdot_\eta \mathbf{w}|}{\|\mathbf{v}\|_\eta \|\mathbf{w}\|_\eta} \in [0, 1] \tag{5.7}$$

$$S_{\text{struct}} = \frac{\|\mathbf{v} \wedge_\eta \mathbf{w}\|_2}{\|\mathbf{v}\|_\eta \|\mathbf{w}\|_\eta} \in [0, 1] \tag{5.8}$$

$$S_{\text{combined}} = \sqrt{S_{\text{func}}^2 + S_{\text{struct}}^2} \in [0, \sqrt{2}] \tag{5.9}$$

$$R_{f/s} = \frac{S_{\text{func}}}{S_{\text{struct}} + \varepsilon} \tag{5.10}$$

where $\|\mathbf{v}\|_\eta = \sqrt{|\mathbf{v} \cdot_\eta \mathbf{v}|}$.

The interpretation is:

- $R_{f/s} > 2.0$: "Functionally similar, structurally different"
- $R_{f/s} < 0.5$: "Structurally similar, functionally different"
- Otherwise: "Balanced: similar in both aspects"

Example 5.2 (Complete decomposition)**.** Using the vectors from the previous example:

$$\begin{aligned} \|\mathbf{v}\|_\eta &= \sqrt{|\mathbf{v} \cdot_\eta \mathbf{v}|} = \sqrt{|(-1)(0.5)^2 + (+1)(0.4)^2|} = \sqrt{|-0.25 + 0.16|} = \sqrt{0.09} = 0.3 \\ \|\mathbf{w}\|_\eta &= \sqrt{|(-1)(0.6)^2 + (+1)(0.3)^2|} = \sqrt{|-0.36 + 0.09|} = \sqrt{0.27} = 0.5196 \end{aligned} \tag{5.11}$$

$$S_{\text{func}} = \frac{|\mathbf{v} \cdot_\eta \mathbf{w}|}{\|\mathbf{v}\|_\eta \|\mathbf{w}\|_\eta} = \frac{0.18}{0.3 \times 0.5196} = \frac{0.18}{0.1559} = 1.155 \tag{5.12}$$

$$S_{\text{struct}} = 0 \quad \text{(no bivector)} \tag{5.13}$$

$$S_{\text{combined}} = 1.155, \quad R_{f/s} = \infty \tag{5.14}$$

Interpretation: $R_{f/s} > 2.0$ indicates "Functionally similar, structurally different" (in this case, $S_{\text{struct}} = 0$ means they are structurally identical in the components considered).

CHAPTER 6

Clifford Rotors and Rotation Angles

6.1 Biological Planes in SGP

Definition 6.1. For each biologically relevant plane defined by the index pair (i, j), the rotation angle between vectors $\mathbf{v}$ and $\mathbf{w}$ is:

$$\theta_{ij}(\mathbf{v}, \mathbf{w}) = \arccos\left(\frac{v_i w_i + v_j w_j}{\sqrt{v_i^2 + v_j^2}\sqrt{w_i^2 + w_j^2}}\right) \times \frac{180}{\pi} \text{ degrees} \tag{6.1}$$

The planes implemented in SGP 13.1v are:

$$\begin{aligned}
\text{hydrophobic} &: (10, 15) \quad \text{N} \to \text{N vs NP} \to \text{NP} \\
\text{charge} &: (0, 5) \quad \text{P}^+ \to \text{P}^+ \text{ vs P}^- \to \text{P}^- \\
\text{opposite_charge} &: (1, 4) \quad \text{P}^+ \to \text{P}^- \text{ vs P}^- \to \text{P}^+ \\
\text{polarity} &: (10, 11) \quad \text{N} \to \text{N vs N} \to \text{NP} \\
\text{charge_transition} &: (2, 8) \quad \text{P}^+ \to \text{N vs N} \to \text{P}^+ \\
\text{opposite_transition} &: (6, 9) \quad \text{P}^- \to \text{N vs N} \to \text{P}^-
\end{aligned} \tag{6.2}$$

Example 6.1 (Angle in the hydrophobic plane)**.** Let:

$$\begin{aligned}
\mathbf{v} &= [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.8, 0, 0, 0, 0, 0.2] \\
\mathbf{w} &= [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.3, 0, 0, 0, 0, 0.7]
\end{aligned} \tag{6.3}$$

In the hydrophobic plane $(10, 15)$:

$$v_{10} = 0.8,\ v_{15} = 0.2,\ w_{10} = 0.3,\ w_{15} = 0.7 \tag{6.4}$$

$$\begin{aligned} \text{numerator} &= v_{10}w_{10} + v_{15}w_{15} = (0.8)(0.3) + (0.2)(0.7) = 0.24 + 0.14 = 0.38 \\ \sqrt{v_{10}^2 + v_{15}^2} &= \sqrt{0.64 + 0.04} = \sqrt{0.68} = 0.8246 \\ \sqrt{w_{10}^2 + w_{15}^2} &= \sqrt{0.09 + 0.49} = \sqrt{0.58} = 0.7616 \end{aligned} \tag{6.5}$$

$$\cos\theta = \frac{0.38}{0.8246 \times 0.7616} = \frac{0.38}{0.628} = 0.605 \tag{6.6}$$

$$\theta = \arccos(0.605) \times \frac{180}{\pi} = 52.8^\circ \tag{6.7}$$

Interpretation: The two proteins have a moderate difference in their mixture of hydrophobic interactions vs hydrogen bonds.

CHAPTER 7

Statistical Framework of SGP

7.1 Group Statistics

Definition 7.1 (Group Centroid)**.** For a group G with N vectors $\{\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(N)}\}$:

$$\boldsymbol{\mu}_G = \frac{1}{N} \sum_{k=1}^{N} \mathbf{v}^{(k)} \tag{7.1}$$

Definition 7.2 (Online Statistics - Welford Algorithm)**.** To compute mean and covariance without storing all vectors, SGP uses the Welford algorithm. For each new vector $\mathbf{x}$:

$$n \leftarrow n + 1 \tag{7.2}$$

$$\delta = \mathbf{x} - \boldsymbol{\mu} \tag{7.3}$$

$$\boldsymbol{\mu} \leftarrow \boldsymbol{\mu} + \frac{\delta}{n} \tag{7.4}$$

$$\delta_2 = \mathbf{x} - \boldsymbol{\mu} \tag{7.5}$$

$$\mathbf{M}_2 \leftarrow \mathbf{M}_2 + \delta \delta_2^T \tag{7.6}$$

The covariance is then $\boldsymbol{\Sigma} = \mathbf{M}_2/(n-1)$ for $n \geq 2$.

Example 7.1 (Centroid calculation with Welford)**.** Let three vectors be processed sequentially:

$$\begin{aligned} \mathbf{v}^{(1)} &= [0.5, 0.5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \\ \mathbf{v}^{(2)} &= [0.6, 0.4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \\ \mathbf{v}^{(3)} &= [0.4, 0.6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \end{aligned} \tag{7.7}$$

After processing all three, the centroid is:

$$\boldsymbol{\mu}_G = \left[\frac{0.5+0.6+0.4}{3}, \frac{0.5+0.4+0.6}{3}, 0, \ldots\right] = [0.5, 0.5, 0, \ldots] \tag{7.8}$$

This is achieved with O(1) memory regardless of N.

7.2 Progressive Reservoir Sampling

Definition 7.3 (Reservoir Sampling)**.** Reservoir sampling is a randomized algorithm for maintaining a representative sample of fixed size M from a stream of data of unknown length N. It guarantees that:

1. Every element in the stream has equal probability M/N of being in the final sample.
2. The sample is unbiased: it does not over-represent early or late elements.
3. The algorithm requires only $O(M)$ memory and $O(1)$ time per element.

The algorithm works as follows:

1. Initialize an empty reservoir of size M.
2. For the first M elements, add each to the reservoir.
3. For the t-th element where $t > M$:
   (a) Generate a random integer $j \sim \text{Uniform}\{0, 1, \ldots, t-1\}$.
   (b) If $j < M$, replace the j-th element in the reservoir with the new element.
   (c) Otherwise, discard the new element.

Theorem 7.1 (Unbiasedness of Reservoir Sampling)**.** For a stream of length N and reservoir size M, the probability that a given element is in the final reservoir is exactly M/N.

Proof. For the t-th element to be in the final reservoir, it must be selected when it arrives (probability M/t) and then not be replaced by any later element k (for $k = t+1, \ldots, N$). For a later element k, the probability that it replaces the t-th element is $1/k$. Therefore, the probability that the t-th element survives from time t to N is:

$$P(\text{survive}) = \prod_{k=t+1}^{N} \left(1 - \frac{1}{k}\right) = \prod_{k=t+1}^{N} \frac{k-1}{k} = \frac{t}{N} \tag{7.9}$$

Thus:

$$P(\text{element } t \text{ in final reservoir}) = \frac{M}{t} \times \frac{t}{N} = \frac{M}{N} \tag{7.10}$$

This holds for all t, confirming that the sample is unbiased. □

Example 7.2 (Reservoir Sampling in SGP)**.** SGP uses reservoir sampling with $M = 10,000$ vectors per group.

For the first 10,000 proteins:

- Each is added to the reservoir.
- Sample size grows from 1 to 10,000.

For the 10,001st protein:

- Generate $j \sim \text{Uniform}\{0, \ldots, 10000\}$.

- If $j < 10000$, replace protein j with the new one.
- Otherwise, discard the new protein.

For the 150,000,000th protein:

- Generate $j \sim \text{Uniform}\{0, \ldots, 149999999\}$.
- If $j < 10000$, replace protein j with the new one.
- Probability of being selected: $10000/150000000 \approx 6.67 \times 10^{-5}$.
- This matches M/N, confirming uniform representativeness.

Result: The final sample of 10,000 proteins is a representative, unbiased sample of the entire 150 million-protein group.

Remark 7.1 (Why 10,000?)**.** The choice $M = 10,000$ balances:

- **Statistical representativeness:** 10,000 is sufficient to capture the diversity of most protein groups.
- **Memory efficiency:** 10,000 vectors $\times$ 16 components $\times$ 8 bytes = 1.28 MB per group.
- **Computational efficiency:** Searching 10,000 vectors is fast ($\sim$0.01 seconds).
- **LSH index size:** 10,000 entries per group for instant hash-based search.

7.3 Adaptive Threshold

Definition 7.4. For group G, compute all within-sample wedge product similarities $\{s_{ij}\}$:

$$\tau_G = \text{percentile}(\{s_{ij}\}, 5\%) \quad \text{(5th percentile)} \tag{7.11}$$

Special case: For groups with a single member ($n = 1$), the 5th percentile is mathematically undefined. In such cases, the adaptive threshold is set to a conservative default value of 0.99, reflecting the expected self-similarity of a single vector with a 1% tolerance margin for numerical precision. This ensures that singleton groups are not incorrectly classified as belonging to larger, more diverse groups.

Example 7.3 (Adaptive threshold calculation)**.** Suppose a group has the following internal similarities (wedge product):

$$\{0.99, 0.98, 0.97, 0.96, 0.95, 0.94, 0.93, 0.92, 0.91, 0.90, 0.89, 0.88, 0.87, 0.86, 0.85\} \tag{7.12}$$

Sorted: $[0.85, 0.86, 0.87, 0.88, 0.89, 0.90, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98, 0.99]$

The 5th percentile corresponds to position $\lceil 0.05 \times 15 \rceil = 1$, i.e., the minimum value:

$$\tau_G = 0.85 \tag{7.13}$$

An external protein is considered similar if its similarity to the centroid is ≥ 0.85.

7.4 Bootstrap Confidence Intervals

Definition 7.5. For B bootstrap iterations[25], sample with replacement[26] from the vector components:

[25] Bootstrap is a statistical technique that involves repeatedly sampling with replacement from the original data to estimate the variability of a measure. It was developed by Bradley Efron in 1979.

[26] Sampling with replacement means that after selecting an element, it is returned to the original set, so it can be selected multiple times.

For $b = 1, \ldots, B$:

$$\begin{aligned} \mathbf{v}_k^{(b)} &= v_{I_b(k)} \quad \text{where } I_b \sim \text{Uniform}\{0, \ldots, 15\} \\ \mathbf{w}_k^{(b)} &= w_{J_b(k)} \end{aligned} \tag{7.14}$$

Compute the wedge product similarity $s_b = \|\mathbf{v}^{(b)} \wedge \mathbf{w}^{(b)}\|_{\text{norm}}$. Then:

$$\hat{s} = \frac{1}{B} \sum_{b=1}^{B} s_b \tag{7.15}$$

$$\hat{\sigma}_s = \sqrt{\frac{1}{B-1} \sum_{b=1}^{B} (s_b - \hat{s})^2} \tag{7.16}$$

CHAPTER 8

Memory-Efficient Processing Architecture

8.1 Streaming Input Processing

Definition 8.1 (Streaming FASTA Reader)**.** SGP reads FASTA files using a Python generator that yields sequences one at a time:

$$\text{for each } (\text{header}, \text{sequence}) \in \text{read\_fasta\_stream}(\text{filepath}) : \tag{8.1}$$

The generator reads the file line-by-line, never loading the entire file into memory. This enables processing of arbitrarily large files (e.g., 149,234,636 sequences, ∼170 GB uncompressed) with constant memory usage.

8.2 Online Statistics with Welford Algorithm

Definition 8.2. Instead of storing all PIM vectors, SGP computes group-level statistics incrementally:

For a group with n vectors, the algorithm maintains:

- Running mean: $\bar{x}_n = \bar{x}_{n-1} + (x_n - \bar{x}_{n-1})/n$
- Running covariance: $M_{2,n} = M_{2,n-1} + (x_n - \bar{x}_{n-1})(x_n - \bar{x}_n)^T$

This requires only O(1) memory (16 + 256 = 272 floats, approximately 2 KB per group).

8.3 Locality-Sensitive Hashing (LSH) for Instant Similarity Search

Locality-Sensitive Hashing (LSH) is a technique that enables $O(1)$ similarity search by hashing similar vectors into the same bucket. SGP implements LSH using a custom hash function based on discretized PIM vectors.

8.3.1 The Hash Function

Definition 8.3 (Hash Function for PIM Vectors)**.** For a tolerance $\delta = 0.001$, discretize[27] the vector $\mathbf{v}$:

[27] Discretizing means rounding a continuous number to a value on a predefined grid. For example, 0.1667 is discretized to 0.166 because the grid has spacing 0.001.

$$\hat{v}_k = \delta \cdot \left\lfloor \frac{v_k}{\delta} \right\rfloor \tag{8.2}$$

The hash key[28] is:

$$h(\mathbf{v}) = \text{SHA256}\left(\bigoplus_{k=0}^{15} \text{format}(\hat{v}_k) \right)_{[:32]} \tag{8.3}$$

where $\oplus$ denotes string concatenation[29] and format produces a fixed-width decimal representation.

8.3.2 The Search Algorithm

Given a query vector $\mathbf{q}$, the LSH search algorithm proceeds as follows:

1. Compute the hash key $h(\mathbf{q})$.
2. Retrieve all vectors from the hash bucket with key $h(\mathbf{q})$.
3. For each retrieved vector $\mathbf{v}$, compute the exact similarity using the wedge product.
4. Return the k most similar vectors.

Theorem 8.1 (Correctness of LSH Search)**.** If $\|\mathbf{v}-\mathbf{w}\|_\infty \leq \delta$, then $h(\mathbf{v}) = h(\mathbf{w})$. Conversely, if $h(\mathbf{v}) = h(\mathbf{w})$, then $\|\mathbf{v} - \mathbf{w}\|_\infty \leq \delta$.

Proof. The hash function discretizes each component to the nearest multiple of δ. Two vectors have the same hash if and only if each component differs by less than δ. This is exactly the condition $\|\mathbf{v} - \mathbf{w}\|_\infty \leq \delta$. □

Example 8.1 (LSH Search in SGP)**.** Suppose we have a database of 150 million proteins indexed using LSH.

For a query protein $\mathbf{q}$:

1. Compute $h(\mathbf{q})$.
2. Suppose only 5 proteins have the same hash.
3. Compare $\mathbf{q}$ with only these 5 proteins using the wedge product.
4. Return the most similar protein.

Instead of 150 million comparisons, we perform only 5 comparisons. Speedup: $150,000,000/5 = 30,000,000\times$ faster.

Remark 8.1 (False Positives and Negatives)**.** LSH is an approximate method:

- **False positives:** Two vectors with the same hash may not be truly similar. This is handled by exact wedge product comparison within the bucket.
- **False negatives:** Two similar vectors with a difference slightly larger than δ may fall into different buckets. This is mitigated by using multiple hash functions or a larger tolerance δ.

In SGP, $\delta = 0.001$ is small enough to minimize false negatives while still providing efficient hashing. Since the exact wedge product is computed within each bucket, false positives are eliminated.

Remark 8.2 (Time Complexity of LSH)**.**

- **Hash computation:** $O(16)$ time per query (constant).
- **Bucket retrieval:** $O(1)$ average time (hash table lookup).

[28] A hash is a function that converts any input (such as a vector) into a fixed-length string. A good hash function produces different strings for different inputs.

[29] Concatenation means joining text strings one after another. For example, concatenating "abc" and "123" gives "abc123".

- **Exact comparison:** $O(b \cdot |\mathcal{K}|)$, where b is the bucket size and $|\mathcal{K}| = 6$ (constant).

Total: $O(1)$ time for instant similarity search, independent of database size.

Remark 8.3 (Memory Requirements of LSH)**.** The LSH index stores:

- One hash key per protein in the sample (10,000 per group).
- The hash key is 32 bytes (SHA256 truncated).
- Total: $10,000 \times 32$ bytes $\times 18$ groups ≈ 5.76 MB.

This is minimal compared to storing the vectors themselves.

8.4 O(1) Similarity Search

Given a query vector $\mathbf{q}$, compute $h(\mathbf{q})$ and retrieve all vectors with the same hash. Two vectors with identical hashes satisfy:

$$\max_k |v_k - w_k| \leq \delta \tag{8.4}$$

8.5 Complete Memory and Time Analysis

Component	**Memory**	**Time Complexity**
Streaming FASTA reader	$O(1)$	$O(N)$
Welford statistics	272 floats (∼2 KB per group)	$O(N)$ per group
Reservoir sampling	$M \times 16$ floats (∼1.28 MB per group)	$O(1)$ per element
LSH index	$M \times 32$ bytes (∼5.76 MB total)	$O(1)$ per search
Centroids	18 vectors (∼2.3 KB total)	$O(G^2)$ for comparisons
Total	∼41 MB	$O(N) + O(G^2)$

CHAPTER 9

Implementation Architecture

9.1 Data Structures

Definition 9.1 (GroupStatistics in SGP)**.** Container for a protein group containing:

- centroid $\boldsymbol{\mu}_G \in \mathbb{R}^{16}$
- covariance matrix $\boldsymbol{\Sigma}_G \in \mathbb{R}^{16\times 16}$
- inverse covariance $\boldsymbol{\Sigma}_G^{-1}$
- adaptive threshold τ_G
- Clifford signature[30] σ_G
- subspace projections $\{p_S\}$
- metric norm and metric sign
- total processed count N_{total}
- sample size stored N_{sample}

9.2 Performance Achieved

With streaming I/O, online statistics, progressive sampling, and LSH, SGP processes 151,066,923 protein sequences in less than 13 hours on standard hardware:

- Processing rate: ∼3,200 sequences per second
- Peak memory usage: ∼500 MB
- Largest file processed: 149,234,636 sequences (∼170 GB uncompressed)
- No GPU acceleration or distributed computing required

[30]The Clifford signature is a numerical summary that captures the most important properties of the vector from the perspective of geometric algebra: its norm, self-reflection, projections, etc.

9.3 Time Complexity

For N proteins in G groups, $d = 16$, $|\mathcal{K}| = 6$, $B = 100$:

- Statistics construction (streaming): $O(Nd^2) \approx O(256N)$ with O(1) memory
- Single protein classification: $O(Gd^2 + GB|\mathcal{K}|) \approx O(856G)$
- Pairwise group comparison: $O(G^2d^2) \approx O(256G^2)$
- Top-k search (LSH): $O(N_{\text{sample}} \cdot |\mathcal{K}| + N_{\text{sample}} \log k) \approx O(6N_{\text{sample}} + N_{\text{sample}} \log k)$ where $N_{\text{sample}} = 10,000$

Example 9.1 (Scalability for 151M sequences)**.** For $N = 151,066,923$ proteins and $G = 18$ groups:

- Statistics construction: $\approx 38.7 \times 10^9$ operations, but streaming requires O(1) memory
- Peak memory: ∼500 MB (only 10,000 samples stored)
- Time: ∼13 hours on Intel Core i5, 16 GB RAM

9.4 What SGP Does and Does Not Do: A Clarification

The SGP program is designed for **efficient functional characterization** of proteins based on polarity patterns, not for sequence alignment or exhaustive pairwise comparison. To avoid common misconceptions, this section explicitly clarifies the scope and limitations of the algorithm.

9.4.1 What SGP Does NOT Do

1. **Does NOT perform sequence alignment.** SGP does not use BLAST, Smith-Waterman, or any other alignment algorithm. It does not compare amino acid sequences directly.
2. **Does NOT compare every protein against every other protein.** This would be $O(N^2)$ and computationally prohibitive for 150 million sequences. Instead, SGP computes group centroids and compares only these centroids.
3. **Does NOT store all protein vectors in memory.** For 150 million sequences with 16 components each, storing all vectors would require ∼19 GB just for the raw data, plus Python overhead (∼30-40 GB total). SGP stores only a representative sample of 10,000 vectors per group (∼1.28 MB per group).
4. **Does NOT require multiple sequence alignments (MSA).** Unlike AlphaFold or other structure prediction tools, SGP does not need evolutionary information from homologous sequences.
5. **Does NOT predict 3D structure.** SGP provides a structure-independent measure of functional similarity based on polarity organization.
6. **Does NOT use GPU acceleration or distributed computing.** The program runs on a standard laptop (Intel Core i5, 16 GB RAM) without special hardware requirements.

9.4.2 What SGP Actually Does

1. **Reads sequences in streaming mode.** The program reads FASTA files line-by-line using a Python generator, never loading entire files into memory. This allows processing of arbitrarily large files (e.g., 149,234,636 sequences, ∼170 GB uncompressed) with constant memory usage.
2. **Computes a 16-component PIM vector for each sequence.** For each protein, the program calculates the frequency of 16 polarity transitions (e.g., $P^+ \to P^+$, $P^+ \to P^-$, etc.) using a sliding window of length 2. This is a simple $O(L)$ operation per sequence, where L is the sequence length.

3. **Updates online statistics for each group.** Using the Welford algorithm, the program maintains the running mean and covariance matrix for each group in $O(1)$ memory. This is done in a single pass through the data.
4. **Stores a representative sample of 10,000 vectors per group.** Using progressive reservoir sampling, the program ensures that the sample is unbiased and representative of the entire group. This sample is used for:
   - Computing group cohesion (internal similarity)
   - Building the LSH hash index for instant similarity search
   - Identifying the top k most similar proteins
5. **Compares groups using centroids.** Instead of comparing all proteins pairwise, SGP computes the centroid (mean vector) for each group and compares centroids using the geometric algebra operators (wedge product, commutator, etc.). For 18 groups, this is only $18 \times 17/2 = 153$ comparisons.
6. **Provides instant similarity search via LSH.** The Locality-Sensitive Hashing (LSH) index enables $O(1)$ retrieval of similar proteins by hashing the 16-component vectors into buckets. A query protein is hashed and only proteins in the same bucket (with tolerance $\delta = 0.001$) are compared.
7. **Computes full statistics for human proteome separately.** The function `compute_lujv_statistics()` reads all 147,506 human proteins from disk again to generate distribution histograms (Figures 2-7). This is a separate, one-time operation that takes ∼30-60 minutes.

9.4.3 Why Processing 150 Million Sequences in 13 Hours is Possible

The key to SGP's speed is that it avoids computationally expensive operations:

- **No pairwise comparisons:** $O(N^2)$ is replaced by $O(N)$ for statistics + $O(G^2)$ for group comparisons.
- **Low dimensionality:** Only 16 dimensions (the 16 polarity transitions), compared to 1024+ dimensions in protein language models.
- **Simple operations:** Each PIM vector computation involves only counting transitions and normalizing, not complex matrix operations or deep learning.
- **Streaming I/O:** No need to load entire files into memory.
- **Online statistics:** Mean and covariance are computed in a single pass.
- **Reservoir sampling:** Only 10,000 vectors per group are stored in RAM.

9.4.4 Performance Comparison

Tool	What it does	Time for 150M seq	Memory
BLAST	Pairwise sequence alignment	Years ($O(N^2)$)	TBs
DIAMOND	Fast sequence alignment	Weeks ($O(N^2)$)	GBs
MMseqs2	Sequence clustering	Days ($O(N \log N)$)	GBs
AlphaFold	3D structure prediction	Impossible (minutes per protein)	GBs
ESM-2	Protein language model	Months (GPU required)	TBs
SGP (v13.1)	**Polarity-based functional analysis**	**∼13 hours ($O(N)$)**	**∼500 MB**

9.4.5 Summary of Key Points

- SGP is a **functional profiling tool**, not a sequence alignment tool.
- SGP processes sequences in **linear time** $O(N)$, not quadratic $O(N^2)$.
- SGP uses **constant memory** $O(1)$ per group, not $O(N)$.
- SGP **does not** store all vectors; it stores a representative sample.
- SGP **does not** compare all proteins pairwise; it compares centroids.
- The 13-hour processing time is **realistic and reproducible** on standard hardware.
- The program is **transparent** about what it processes and what it samples.

9.4.6 Verification of Complete Processing

The program includes built-in verification to confirm that all sequences were processed:

1. **ProcessingTracker:** Monitors total sequences read, valid PIM profiles, and processing rate in real time.
2. **Final summary:** Reports total sequences processed per group and overall.
3. **GroupStatistics:** Stores `total_processed` (all sequences) and `n_samples` (valid PIM profiles) separately.
4. `compute_lujv_statistics()` reads all human sequences from disk again to generate histograms, confirming the data is available.

The tracker output at the end of execution shows:

```
 RESUMEN GLOBAL DE PROCESAMIENTO
================================================================================
Tiempo total: 12:47:32
Total secuencias leídas: 151,066,923
Total PIM válidos: 128,456,789
Total rechazados: 22,610,134
Tasa de validez: 85.03%
Total bytes procesados: 185.23 GB
Velocidad promedio: 3,281 seq/s
```

This confirms that all 151,066,923 sequences were processed, even though only 10,000 per group were stored in RAM for subsequent comparisons.

CHAPTER 10

Conclusion

The **Specular-Grassmann-PIM (SGP) 13.1v** program provides a complete geometric algebra formulation for protein interaction analysis with memory-efficient processing for massive datasets. Key innovations include:

1. Biological metric signature η that distinguishes beneficial from detrimental interactions
2. Complete geometric product $v *_\eta w$ unifying functional and structural similarity
3. Subspace projection via interior product $\lrcorner$
4. Specular reflection to detect charge complementarity
5. Clifford rotors to measure conformational changes in biological planes
6. Commutator and anticommutator to measure non-commutativity and metric symmetry
7. Statistical validation via bootstrap[31]
8. $O(1)$ similarity search via LSH[32]
9. Streaming I/O for processing arbitrarily large datasets (151M+ sequences)
10. Online statistics via Welford algorithm for O(1) memory usage
11. Progressive reservoir sampling for memory-efficient storage
12. Processing of 151M sequences in <13 hours on standard hardware with ∼500 MB peak memory

The implementation processes hundreds of millions of proteins while providing mathematically rigorous geometric interpretations of protein-protein interactions and functional relationships.

[31] Bootstrap is a technique that allows estimating the uncertainty of a statistical measure without making strong assumptions about the data distribution.

[32] LSH stands for *Locality-Sensitive Hashing*, a technique that groups similar items into the same hash bucket for instant search.